

```

%-----
%The menu page
%-----
clear; %Clear all variables
clc; %Clear the command window
close all; % Close all opened figures

% note to TAs, Jonathon's portion of the game is mostly the custom sprites
% and graphics, so the menu screens, as well as all of the questions, and
% taking in mouse input.
% Alex's portion is the puzzle game, with all of the logic attached to it,
% including taking in keyboard input.

gameContinue = true; % gameContinue is the variable that controls the game loop; as long as it is true, the menu page will continue to be displayed.

while gameContinue %This is the outer loop that allow the main menu to run all the time, no matter what the user is currently at.
    close all %Close all figures before each game.

clean_scene = simpleGameEngine('retro_pack.png',16,16,1,[220,187,94]);

%initilaizng
pushright = 0;
pushleft = 0;
pushdown = 0;
pushup = 0;
sprite_contento = [555:560];

row = 24;
col = 24;
% 159 = bad guy
% preallocate space because apparently my code runs faster
sprite_battle = zeros(row, col);
%important to establish and initialize each level's bricks
excludedrow = [5,5,5,5,5,5,5,5,5,8,8,8,4,13,13,13,13,13,15,15,15,15,15,15,15,15,15,18,19,20,21,22,23,12,15,18,13,16,19,20];
excludedcol = [3,4,5,7,8,9,10,11,12,14,12,12,12,14,1,2,3,4,5,6,7,8,9,2,3,4,5,6,7,8,9,10,11,17,17,17,17,17,16, 19, 22, 23,18,20,21];

% initializes each levels spills
spill_row = [4,14,22,15,18];
spill_col = [4,2,23,20,8];

%initializes where the soap will spawn
soaprow = [8]; %591
soapcol = [6];

currantdialogue = 0;
dialoguebox = [1,1];
% initializing the random dirt pattern into a separate array for use in the
% UDF.
for row = 3:24
    for col = 1:24
        dirtpattern(row, col) = randi([2,5]);
    end
end

%initialize player position
sprite_battle(10,10) = 30;
xcord = 10;
ycord = 10;
% call to population, to draw to the current array, what bricks, and
% current dirt pattern generated previously.
sprite_battle = population(sprite_battle, excludedrow, excludedcol, dirtpattern);
cleansed = 0;

chaponecontinue = 0;

rowup = 0;
rowdown = 0;
colright = 0;
colleft = 0;

%The code for the Menu Page
%The sprite sheet is 424*424, so 424/5=85
%magnification = 8
%there are 5 columns in the sprite sheet;
%the spite sheet is 5*5, so the max number of grid is 25

SDPgame = simpleGameEngine("Menu.png", 84, 84, 8); %all the size informations

%Assigning icons in the sprite sheet to variables

```

```

BackGround1 = 1;
BackGround2 = 2;
BackGround3 = 3;
BackGround4 = 4;
BackGround5 = 5;
BackGround6 = 6;
BackGround7 = 7;
BackGround8 = 8;
BackGround9 = 9;
BackGround10 = 10;
BackGround11 = 11;
BackGround12 = 12;
BackGround13 = 13;
BackGround14 = 14;
BackGround15 = 15;
BackGround16 = 16;
BackGround17 = 17;
BackGround18 = 18;
BackGround19 = 19;
BackGround20 = 20;
BackGround21 = 21;
Start = 22;
BackGround23 = 23;
Exit = 24;
BackGround25 = 25;

%Set up the actual scene
Rows=5; Columns=5; %The size of the figure output, which is 5*5
Box = ones(Rows, Columns); %Set all 25 boxes in the figure as the default background (box 1), which is why we use "ones".

Box(1,1) = BackGround1;
Box(1,2) = BackGround2;
Box(1,3) = BackGround3;
Box(1,4) = BackGround4;
Box(1,5) = BackGround5;
Box(2,1) = BackGround6;
Box(2,2) = BackGround7;
Box(2,3) = BackGround8;
Box(2,4) = BackGround9;
Box(2,5) = BackGround10;
Box(3,1) = BackGround11;
Box(3,2) = BackGround12;
Box(3,3) = BackGround13;
Box(3,4) = BackGround14;
Box(3,5) = BackGround15;
Box(4,1) = BackGround16;
Box(4,2) = BackGround17;
Box(4,3) = BackGround18;
Box(4,4) = BackGround19;
Box(4,5) = BackGround20;
Box(5,1) = BackGround21;
Box(5,2) = Start; %Start Button
Box(5,3) = BackGround23;
Box(5,4) = Exit; %Exit Button
Box(5,5) = BackGround25;

drawScene(SDPgame, Box); %draw out the figure
xlabel('Click Start/Exit or DOUBLE Press s/e; You can choose to play the game or quit the game, and the game title is featured')
title('Main Menu'); %The title of the created figure so I know which figure is for which

choice = ""; %Set the choice to be initially empty
while choice == "" %"" because We want the code inside the loop to continue for execute only if the choice is empty
    %This part of the code is for the user's mouse input
    [r,c,b] = getMouseInput(SDPgame); %Row = r, Column = c, and b is which key you click with by your mouse
    if b==1 %We use b==1 because the correct feedback will only occur when the user clicks the button/choice with the left mouse button.
        if r==5 && c==2
            choice="START"; %The START icon is located on (5,2), which is why the code is r==5 && c==2. r means row, c means column
        end
        if r==5 && c==4
            choice="EXIT"; %The EXIT icon is located on (5,4), which is why the code is r==5 && c==4
        end
    end

    %This part of the code is for the user's keyboard input
    while choice==" " %"" because We want the code inside the loop to continue for execute only if the choice is empty. Basically, if the keyboard button is not
        key = getKeyboardInput(SDPgame); % As long as k is one of the keys I specify, the user's choice can be assigned the correct value.
        if strcmp(key,"s") %If the user DOUBLE clicked s, the game will start
            choice="START";
        end

        if strcmp(key,"e") %If the user DOUBLE clicked s, they will exit the game menu
            choice="EXIT";
        end
    end
end
end
end

```

```

%Feedback for the user when they "made" a choice so they know what they are
%doing when the clicked the keys
if choice=="START"
    title("Start selected");
    xlabel("Loading the game...");
    pause(2); %wait for 2 seconds
elseif choice=="EXIT"
    title("Exit selected");
    xlabel("Closing...");
    pause(2);
    close all %If the user clicked the EXIT button, then it will close all figures
    break %Exit the main loop, which will end the entire game, basically means the user quit the game
end

```

```

%=====
%Chapter 1 / 3
%=====

```

```

% jonathan
%The code for the Chapter 1 Page
%The sprite sheet is 1024*1024, so 1024/5=204
%magnification = 8
%there are 5 columns and 5 rows in the sprite sheet;
%the spite sheet is 5*5, so the max number of grid is 25

```

```

SDPgame = simpleGameEngine("Question 1 page.png", 204, 204, 8); %all the size informations

```

```

%Assigning icons in the sprite sheet to variables

```

```

BackGround1 = 1;
BackGround2 = 2;
BackGround3 = 3;
Choice1 = 4;
Choice2 = 5;
BackGround6 = 6;
BackGround7 = 7;
BackGround8 = 8;
Choice3 = 9;
Choice4 = 10;
BackGround11 = 11;
BackGround12 = 12;
BackGround13 = 13;
BackGround14 = 14;
BackGround15 = 15;
Choice5 = 16;
Choice6 = 17;
BackGround18 = 18;
BackGround19 = 19;
BackGround20 = 20;
Choice7 = 21;
Choice8 = 22;
BackGround23 = 23;
BackGround24 = 24;
BackGround25 = 25;

```

```

%Set up for chapter 1

```

```

Rows=5; Columns=5; %The size of the chapter 1, which is 5*5

```

```

Box = ones(Rows, Columns); %Set all 25 boxes in the figure as the default background (box 1), which is why we use "ones".

```

```

Box(1,1) = BackGround1;
Box(1,2) = BackGround2;
Box(1,3) = BackGround3;
Box(1,4) = Choice1; %Choice 1 is for question 1
Box(1,5) = Choice2; %Choice 2 is for question 1
Box(2,1) = BackGround6;
Box(2,2) = BackGround7;
Box(2,3) = BackGround8;
Box(2,4) = Choice3; %Choice 3 is for question 1
Box(2,5) = Choice4; %Choice 4 is for question 1
Box(3,1) = BackGround11;
Box(3,2) = BackGround12;
Box(3,3) = BackGround13;
Box(3,4) = BackGround14;
Box(3,5) = BackGround15;
Box(4,1) = Choice5; %Choice 5 is for question 2
Box(4,2) = Choice6; %Choice 6 is for question 2
Box(4,3) = BackGround18;
Box(4,4) = BackGround19;

```

```

Box(4,5) = BackGround20;
Box(5,1) = Choice7; %Choice 7 is for question 2
Box(5,2) = Choice8; %Choice 8 is for question 2
Box(5,3) = BackGround23;
Box(5,4) = BackGround24;
Box(5,5) = BackGround25;

drawScene(clean_scene,sprite_battle);

chaponecontinue = 0;

while chaponecontinue == 0

    key = getKeyboardInput(clean_scene);

    if currantdialogue == 2

        if dialoguebox(1) == 1
            currantdialoguestring = "Hello it is me the evil big oil CEO. I am going to give you a hint. When fossil fuels like coal are burned and processed, they rele
uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));

        dialoguebox(1) = 0;
        currantdialoguestring = " Coal is made when organic carbon rich plant material gets compressed underground. We can then mine this and burn it. ";
        end

    elseif currantdialogue == 4
        if dialoguebox(2) == 1
            uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));
            dialoguebox(2) = 0;
        end
    end
    %drawScene(clean_scene, sprite_battle);

    for f = 1:length(excludedcol)
    if((soapcol(1) == excludedcol(f) && soaprow(1) == excludedrow(f)) || ((soapcol(1) <= 1 || soapcol(1) >= 24) || (soaprow(1) <= 3 || soaprow(1) >= 24)))
        soapcol = 6;
        soaprow = 5;
    end
    end

    if(cleansed == 0)
    % placing every spill in the matrix
        for j = 1:length(spill_row)
            sprite_battle(spill_row(j), spill_col(j)) = 559;
        end
        %checking if soap loc = spill loc, if true, set that one to 1.
        for g = 1:length(spill_row)
            if(soaprow(1) == spill_row(g) && soapcol == spill_col(g))
                spill_row(g) = 1;
                spill_col(g) = 1;
                currantdialogue = currantdialogue + 1;
            end
        end
        %if sum is length, the entire array is 1
    if(sum(spill_row) == length(spill_row))
        cleansed = 1;
    end
    end

    % checking input from wasd, but also other int that consistently checks if
    % you can go upwards, and if you cant, rowup == 1
    % w = up, so remove a row from your characters cords.
    if strcmp(key,"w") && (rowup == 0)
        xcord = xcord - 1;
        sprite_battle(xcord, ycord) = 30;
        %pushup means if the player is in contact with the soap,
        % pushup == 1 means capable of pushing it up
        if(pushup == 1)
            soaprow(1) = soaprow(1)-1;
        end
        %redraw it with new soap cols
        sprite_battle(soaprow, soapcol) = 591;
        drawScene(clean_scene,sprite_battle);
        elseif strcmp(key, 'a') && (colleft == 0)
            ycord = ycord - 1;
            sprite_battle(xcord, ycord) = 30;

        if(pushleft == 1)
            soapcol(1) = soapcol(1)-1;
        end
        sprite_battle(soaprow, soapcol) = 591;
        drawScene(clean_scene,sprite_battle);
        elseif strcmp(key, 's') && (rowdown == 0)

```

```

        xcord = xcord + 1;
        sprite_battle(xcord, ycord) = 30;

    if(pushdown == 1)
        soaprow(1) = soaprow(1)+1;
    end
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
    elseif strcmp(key, 'd') && (colright == 0)
        ycord = ycord + 1;
        sprite_battle(xcord, ycord) = 30;

    if(pushright == 1)
        soapcol(1) = soapcol(1)+1;
    end
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
end

%reset all of the collision checkers at the end of a complete run
rowdown = 0;
rowup = 0;
colleft = 0;
colright = 0;

pushup = 0;
pushdown = 0;
pushleft = 0;
pushright = 0;

for g = 1:length(excludedrow)

    %fprintf(' Row: %4.0f Col: %4.0f \n Up: %4.0f Down: %4.0f Left: %4.0f Right: %4.0f \n pushup: %4.0f pushdown %4.0f pushleft: %4.0f pushright: %4.0f', xcord,

    %checking if the players row = blocks row + 1, which means player is below
    %block, so prevent from moving up. also check if youre in the same column,
    %so it doesnt mess anything else up
    if(xcord == (excludedrow(g) + 1) && (ycord == (excludedcol(g))))
        rowup = 1;
    else
        if(xcord == (soaprow(1) + 1) && (ycord == (soapcol(1))))

            pushup = 1;

        end
    end
    if(xcord == (excludedrow(g) - 1) && (ycord == (excludedcol(g))))
        rowdown = 1;
    else
        if(xcord == (soaprow(1) - 1) && (ycord == (soapcol(1))))
            pushdown = 1;
        end
    end
    if(ycord == (excludedcol(g) + 1) && (xcord == (excludedrow(g))))
        colleft = 1;
    elseif(ycord == (soapcol(1) + 1) && (xcord == (soaprow(1))))
        pushleft = 1;
    end
    if(ycord == (excludedcol(g) - 1) && (xcord == (excludedrow(g))))
        colright = 1;
    elseif(ycord == (soapcol(1) - 1) && (xcord == (soaprow(1))))
        pushright = 1;
    end
end
%this is just checking borders
if(xcord <= 4)
    rowup = 1;
else

end
if(xcord >= 23)
    rowdown = 1;
else

end
if(ycord <= 2)
    colleft = 1;
else

end
if(ycord >= 23)
    colright = 1;
else

end
end
end

```

```

%final drawing
sprite_battle = population(sprite_battle, excludedrow, excludedcol, dirtpattern);

%tracks if spills are complete
%b tracks while loop for the game
if(cleansed == 1)
    chaponecontinue = 500;

end

end

drawScene(SDPgame, Box); %draw out Chapter 1's figure
xlabel("Click on the right choice for both question to go to the next chapter! If you choose the wrong choice, you will need to replay the game and lose all you
title("Chapter 1 / 3"); %Shows what chapter the user is on, with progress

%These are the correct choices
correctChoiceCh1 = [1, 4; 4, 1]; %The correct choice is at (1,4) and (4,1), it is passed as a parameter into user-defined function 'userPlay Chapter Level' to

% Call the userPlayChapterLevel function to start Chapter 1
% Return values: Win = true (the user pass the chapter) or false (the user failes to pass the chapter)
Win = userPlayChapterLevel(SDPgame, correctChoiceCh1, 'Chapter 1/3'); %Game object SDPgame, correct coordinates correctChoiceCh1, chapter name 'Chapter 1/3'

if Win == false % If Win == false, it means the user failed in Chapter 1.
    close all;%Close all game windows
    break; % Exit the script flow
end

%=====
%Chapter 2 / 3
%=====

%The code for the Chapter 2 Page
%The sprite sheet is 1024*1024, so 1024/5=204
%magnification = 8
%there are 5 columns and 5 rows in the sprite sheet;
%the spite sheet is 5*5, so the max number of grid is 25

SDPgame = simpleGameEngine("Question 2 page.png", 204, 204, 8); %all the size informations

%Assigning icons in the sprite sheet to variables
BackGround1 = 1;
BackGround2 = 2;
BackGround3 = 3;
Choice1 = 4;
Choice2 = 5;
BackGround6 = 6;
BackGround7 = 7;
BackGround8 = 8;
Choice3 = 9; %Correct choice for Q1
Choice4 =10;
BackGround11 =11;
BackGround12 = 12;
BackGround13 = 13;
BackGround14 = 14;
BackGround15 = 15;
Choice5 = 16;
Choice6 = 17; %Correct Choice for Q2
BackGround18 = 18;
BackGround19 = 19;
BackGround20 = 20;
Choice7 = 21;
Choice8 = 22;
BackGround23 = 23;
BackGround24 = 24;
BackGround25 = 25;

%Set up for chapter 2
Rows=5; Columns=5; %The size of the chapter 2, which is 5*5
Box = ones(Rows, Columns); %Set all 25 boxes in the figure as the default background (box 1), which is why we use "ones".

Box(1,1) = BackGround1;
Box(1,2) = BackGround2;
Box(1,3) = BackGround3;
Box(1,4) = Choice1; %Choice 1 is for question 1
Box(1,5) = Choice2; %Choice 2 is for question 1
Box(2,1) = BackGround6;
Box(2,2) = BackGround7;
Box(2,3) = BackGround8;

```

```

Box(2,4) = Choice3; %Choice 3 is for question 1
Box(2,5) = Choice4; %Choice 4 is for question 1
Box(3,1) = BackGround11;
Box(3,2) = BackGround12;
Box(3,3) = BackGround13;
Box(3,4) = BackGround14;
Box(3,5) = BackGround15;
Box(4,1) = Choice5; %Choice 5 is for question 2
Box(4,2) = Choice6; %Choice 6 is for question 2
Box(4,3) = BackGround18;
Box(4,4) = BackGround19;
Box(4,5) = BackGround20;
Box(5,1) = Choice7; %Choice 7 is for question 2
Box(5,2) = Choice8; %Choice 8 is for question 2
Box(5,3) = BackGround23;
Box(5,4) = BackGround24;
Box(5,5) = BackGround25;

excludedrow = 7 * ones(1,19);
excludedrow = [excludedrow, 9*ones(1,20)]
excludedrow = [excludedrow, 11*ones(1,21)]
excludedrow = [excludedrow, 4, 4,5,6]
excludedcol = [2:2, 4:7 9:21 23 1:4,6:21 1:21, 8, 13,13,13 ];
length(excludedrow)
length(excludedcol)
excludedrow
excludedcol
% initializes each levels spills
spill_row = [5,5,5,10,23];
spill_col = [10,4,15,2,2];
cleansed = 0;
currantdialogue = 0;
dialoguebox = [1,1];
    chaptwocontinue = 0;
    while chaptwocontinue == 0

        key = getKeyboardInput(clean_scene);

if currantdialogue == 2

    if dialoguebox(1) == 1
        currantdialoguestring = "When I burn petroleum to spin turbines, it releases greenhouse gasses with various negative effects on the environment."
        uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));

dialoguebox(1) = 0;
currantdialoguestring = "In my generator factories, petroleum is used as fuel to create energy to power other technology.";
        end

elseif currantdialogue == 4
    if dialoguebox(2) == 1
        uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));
        dialoguebox(2) = 0;
    end
end
%drawScene(clean_scene, sprite_battle);

for f = 1:length(excludedcol)
if((soapcol(1) == excludedcol(f) && soaprow(1) == excludedrow(f)) || ((soapcol(1) <= 1 || soapcol(1) >= 24) || (soaprow(1) <= 3 || soaprow(1) >= 24)))
    soapcol = 8;
    soaprow = 5;
end
end

if(cleansed == 0)
% placing every spill in the matrix
    for j = 1:length(spill_row)
        sprite_battle(spill_row(j), spill_col(j)) = 559;
    end
    %checking if soap loc = spill loc, if true, set that one to 1.
    for g = 1:length(spill_row)
        if(soaprow(1) == spill_row(g) && soapcol == spill_col(g))
            spill_row(g) = 1;
            spill_col(g) = 1;
            currantdialogue = currantdialogue + 1;
        end
    end
    %if sum is length, the entire array is 1
if(sum(spill_row) == length(spill_row))
    cleansed = 1;
end
end

% checking input from wasd, but also other int that consistently checks if
% you can go upwards, and if you cant, rowup == 1

```

```

% w = up, so remove a row from your characters cords.
if strcmp(key,"w") && (rowup == 0)
    xcord = xcord - 1;
sprite_battle(xcord, ycord) = 30;
%pushup means if the player is in contact with the soap,
% pushup == 1 means capable of pushing it up
if(pushup == 1)
soaprow(1) = soaprow(1)-1;
end
%redraw it with new soap cols
sprite_battle(soaprow, soapcol) = 591;
drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 'a') && (colleft == 0)
    ycord = ycord - 1;
sprite_battle(xcord, ycord) = 30;

if(pushleft == 1)
soapcol(1) = soapcol(1)-1;
end
    sprite_battle(soaprow, soapcol) = 591;
drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 's') && (rowdown == 0)
    xcord = xcord + 1;
sprite_battle(xcord, ycord) = 30;

if(pushdown == 1)
soaprow(1) = soaprow(1)+1;
end
    sprite_battle(soaprow, soapcol) = 591;
drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 'd') && (colright == 0)
    ycord = ycord + 1;
sprite_battle(xcord, ycord) = 30;

if(pushright == 1)
soapcol(1) = soapcol(1)+1;
end
    sprite_battle(soaprow, soapcol) = 591;
drawScene(clean_scene, sprite_battle);
end

%reset all of the collision checkers at the end of a complete run
rowdown = 0;
rowup = 0;
colleft = 0;
colright = 0;

pushup = 0;
pushdown = 0;
pushleft = 0;
pushright = 0;

for g = 1:length(excludedrow)

    %fprintf(' Row: %4.0f Col: %4.0f \n Up: %4.0f Down: %4.0f Left: %4.0f Right: %4.0f \n pushup: %4.0f pushdown %4.0f pushleft: %4.0f pushright: %4.0f', xcord,

    %checking if the players row = blocks row + 1, which means player is below
    %block, so prevent from moving up. also check if youre in the same column,
    %so it doesnt mess anything else up
    if(xcord == (excludedrow(g) + 1) && (ycord == (excludedcol(g))))
        rowup = 1;
    else
        if(xcord == (soaprow(1) + 1) && (ycord == (soapcol(1))))

            pushup = 1;

        end
    end
    if(xcord == (excludedrow(g) - 1) && (ycord == (excludedcol(g))))
        rowdown = 1;
    else
        if(xcord == (soaprow(1) - 1) && (ycord == (soapcol(1))))
            pushdown = 1;
        end
    end
    if(ycord == (excludedcol(g) + 1) && (xcord == (excludedrow(g))))
        colleft = 1;
    elseif(ycord == (soapcol(1) + 1) && (xcord == (soaprow(1))))
        pushleft = 1;
    end
    if(ycord == (excludedcol(g) - 1) && (xcord == (excludedrow(g))))
        colright = 1;
    elseif(ycord == (soapcol(1) - 1) && (xcord == (soaprow(1))))
        pushright = 1;
    end
end
end

```

```

%this is just checking borders
if(xcord <= 4)
    rowup = 1;
else

end

if(xcord >= 23)
    rowdown = 1;
else

end

if(ycord <= 2)
    colleft = 1;
else

end

if(ycord >= 23)
    colright = 1;
else

end

end

%final drawing
sprite_battle = population(sprite_battle, excludedrow, excludedcol, dirtpattern);

%tracks if spills are complete
%b tracks while loop for the game
if(cleansed == 1)
    chaptwocontinue = 500;
end

end

drawScene(SDPgame, Box); %draw out Chapter 2's figure
xlabel("Click on the right choice for both question to go to the next chapter! If you choose the wrong choice, you will need to replay the game and lose all you
title("Chapter 2 / 3"); %Shows what chapter the user is on, with progress

%These are the correct choices
correctChoiceCh2 = [2, 4; 4, 2]; %The correct choice is at (2,4) and (4,2), it is passed as a parameter into user-defined function 'userPlay Chapter Level' to

% Call the userPlayChapterLevel function to start Chapter 2
% Return values: Win = true (the user pass the chapter) or false (the user failes to pass the chapter)
Win = userPlayChapterLevel(SDPgame, correctChoiceCh2, 'Chapter 2/3'); %Game object SDPgame, correct coordinates correctChoiceCh2, chapter name 'Chapter 2/3'

if Win == false
    close all;
    break;
end

%=====
%Chapter 3 / 3
%=====

%The code for the Chapter 3 Page
%The sprite sheet is 1024*1024, so 1024/5=204
%maginification = 8
%there are 5 columns and 5 rows in the sprite sheet;
%the spite sheet is 5*5, so the max number of grid is 25

SDPgame = simpleGameEngine("Question 3 page.png", 204, 204, 8); %all the size informations

%Assigning icons in the sprite sheet to variables
BackGround1 = 1;
BackGround2 = 2;
BackGround3 = 3;
Choice1 = 4;
Choice2 = 5;
BackGround6 = 6;
BackGround7 = 7;
BackGround8 = 8;
Choice3 = 9;
Choice4 = 10; %Correct choice for Q1
BackGround11 = 11;
BackGround12 = 12;
BackGround13 = 13;
BackGround14 = 14;
BackGround15 = 15;
Choice5 = 16; %Correct Choice for Q2
Choice6 = 17;

```

```

BackGround18 = 18;
BackGround19 = 19;
BackGround20 = 20;
Choice7 = 21;
Choice8 = 22;
BackGround23 = 23;
BackGround24 = 24;
BackGround25 = 25;

%Set up for chapter 3
Rows=5; Columns=5; %The size of the chapter 3, which is 5*5
Box = ones(Rows, Columns); %Set all 25 boxes in the figure as the default background (box 1), which is why we use "ones".

Box(1,1) = BackGround1;
Box(1,2) = BackGround2;
Box(1,3) = BackGround3;
Box(1,4) = Choice1; %Choice 1 is for question 1
Box(1,5) = Choice2; %Choice 2 is for question 1
Box(2,1) = BackGround6;
Box(2,2) = BackGround7;
Box(2,3) = BackGround8;
Box(2,4) = Choice3; %Choice 3 is for question 1
Box(2,5) = Choice4; %Choice 4 is for question 1
Box(3,1) = BackGround11;
Box(3,2) = BackGround12;
Box(3,3) = BackGround13;
Box(3,4) = BackGround14;
Box(3,5) = BackGround15;
Box(4,1) = Choice5; %Choice 5 is for question 2
Box(4,2) = Choice6; %Choice 6 is for question 2
Box(4,3) = BackGround18;
Box(4,4) = BackGround19;
Box(4,5) = BackGround20;
Box(5,1) = Choice7; %Choice 7 is for question 2
Box(5,2) = Choice8; %Choice 8 is for question 2
Box(5,3) = BackGround23;
Box(5,4) = BackGround24;
Box(5,5) = BackGround25;

excludedrow = [4:10, 12:23, 4:7, 9:23, 4:17, 19:23];
excludedcol = [12* ones(1,19), 5* ones(1,19), 20* ones(1,19)];
% initializes each levels spills
spill_row = [6,18,12,15,8,21];
spill_col = [3,7,10,16,18,22];

cleansed = 0;

chaphthreecontinue = 0;
currantdialogue = 0;
dialoguebox = [1,1];

while chaphthreecontinue == 0

    key = getKeyboardInput(clean_scene);

if currantdialogue == 2

    if dialoguebox(1) == 1
        currantdialoguestring = "What! Wind turbines? The only negative environmental impacts of wind turbines is the disturbance to wildlife, but it also costs a lot";
        uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));

    dialoguebox(1) = 0;
    currantdialoguestring = "Wind energy is generated by wind turning the turbines which by a complicated electromagnetic process produces electricity. Thank you for cleaning";
    end

elseif currantdialogue == 4
    if dialoguebox(2) == 1
        uiwait(msgbox(currantdialoguestring,"Message from CEO","help"));
        dialoguebox(2) = 0;
    end
end
%drawScene(clean_scene, sprite_battle);

for f = 1:length(excludedcol)
if((soapcol(1) == excludedcol(f) && soaprow(1) == excludedrow(f)) || ((soapcol(1) <= 1 || soapcol(1) >= 24) || (soaprow(1) <= 3 || soaprow(1) >= 24)))
    soapcol = 8;
    soaprow = 5;
end
end

if(cleansed == 0)
% placing every spill in the matrix
for j = 1:length(spill_row)
    sprite_battle(spill_row(j), spill_col(j)) = 559;
end

```

```

end
%checking if soap loc = spill loc, if true, set that one to 1.
for g = 1:length(spill_row)
    if(soaprow(1) == spill_row(g) && soapcol == spill_col(g))
        spill_row(g) = 1;
        spill_col(g) = 1;
        currantdialogue = currantdialogue + 1;
    end
end
%if sum is length, the entire array is 1
if(sum(spill_row) == length(spill_row))
    cleansed = 1;
end
end

% checking input from wasd, but also other int that consistently checks if
% you can go upwards, and if you cant, rowup == 1
% w = up, so remove a row from your characters cords.
if strcmp(key,"w") && (rowup == 0)
    xcord = xcord - 1;
    sprite_battle(xcord, ycord) = 30;
    %pushup means if the player is in contact with the soap,
    % pushup == 1 means capable of pushing it up
    if(pushup == 1)
        soaprow(1) = soaprow(1)-1;
    end
    %redraw it with new soap cols
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 'a') && (colleft == 0)
    ycord = ycord - 1;
    sprite_battle(xcord, ycord) = 30;

if(pushleft == 1)
    soapcol(1) = soapcol(1)-1;
end
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 's') && (rowdown == 0)
    xcord = xcord + 1;
    sprite_battle(xcord, ycord) = 30;

if(pushdown == 1)
    soaprow(1) = soaprow(1)+1;
end
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
elseif strcmp(key, 'd') && (colright == 0)
    ycord = ycord + 1;
    sprite_battle(xcord, ycord) = 30;

if(pushright == 1)
    soapcol(1) = soapcol(1)+1;
end
    sprite_battle(soaprow, soapcol) = 591;
    drawScene(clean_scene, sprite_battle);
end

%reset all of the collision checkers at the end of a complete run
rowdown = 0;
rowup = 0;
colleft = 0;
colright = 0;

pushup = 0;
pushdown = 0;
pushleft = 0;
pushright = 0;

for g = 1:length(excludedrow)

    %fprintf(' Row: %4.0f Col: %4.0f \n Up: %4.0f Down: %4.0f Left: %4.0f Right: %4.0f \n pushup: %4.0f pushdown %4.0f pushleft: %4.0f pushright: %4.0f', xcord,

%checking if the players row = blocks row + 1, which means player is below
%block, so prevent from moving up. also check if youre in the same column,
%so it doesnt mess anything else up
if(xcord == (excludedrow(g) + 1) && (ycord == (excludedcol(g))))
    rowup = 1;
else
    if(xcord == (soaprow(1) + 1) && (ycord == (soapcol(1))))

```

```

pushup = 1;

end
end
if(xcord == (excludedrow(g) - 1) && (ycord == (excludedcol(g))))
rowdown = 1;
else
    if(xcord == (soaprow(1) - 1) && (ycord == (soapcol(1))))
pushdown = 1;
end
end
if(ycord == (excludedcol(g) + 1) && (xcord == (excludedrow(g))))
colleft = 1;
elseif(ycord == (soapcol(1) + 1) && (xcord == (soaprow(1))))
pushleft = 1;
end
if(ycord == (excludedcol(g) - 1) && (xcord == (excludedrow(g))))
colright = 1;
elseif(ycord == (soapcol(1) - 1) && (xcord == (soaprow(1))))
pushright = 1;
end
end
end
%this is just checking borders
if(xcord <= 4)
rowup = 1;
else

end
if(xcord >= 23)
rowdown = 1;
else

end
if(ycord <= 2)
colleft = 1;
else

end
if(ycord >= 23)
colright = 1;
else

end

end

%final drawing
sprite_battle = population(sprite_battle, excludedrow, excludedcol, dirtpattern);

%tracks if spills are complete
%b tracks while loop for the game
if(cleansed == 1)
    chapthreecontinue = 500;

end

end

drawScene(SDPgame, Box); %draw out Chapter 2's figure
xlabel("Click on the right choice for both question to master the Game!!! If you choose the wrong choice, you will need to replay the game and lose all your pro
title('Chapter 3 / 3'); %Shows what chapter the user is on, with progress

%These are the correct choices
correctChoiceCh3 = [2, 5; 4, 1]; %The correct choice is at (2,4) and (4,2), it is passed as a parameter into user-defined function 'userPlay Chapter Level' to

% Call the userPlayChapterLevel function to start Chapter 3
% Return values: Win = true (the user pass the chapter) or false (the user failes to pass the chapter)
Win = userPlayChapterLevel(SDPgame, correctChoiceCh3, 'Chapter 3/3'); %Game object SDPgame, correct coordinates correctChoiceCh2, chapter name 'Chapter 2/3'

if Win == false
    close all;
    break;
end

%-----
%The Endgame page
%-----

close all;

%The code for the Endgame Page
%The sprite sheet is 424*424, so 424/5=85
%magnification = 8
%there are 5 columns in the sprite sheet;
%the spite sheet is 5*5, so the max number of grid is 25

```

```

SDPgame = simpleGameEngine("Endgame Background.png", 84, 84, 8); %all the size informations

%Assigning icons in the sprite sheet to variables
BackGround1 = 6;
BackGround2 = 7;
BackGround3 = 8;
BackGround4 = 9;
BackGround5 = 10;
BackGround6 = 11;
BackGround7 = 12;
BackGround8 = 13;
BackGround9 = 14;
BackGround10 = 15;
BackGround11 = 16;
BackGround12 = 18;
BackGround13 = 20;
Replay = 17;
Exit = 19;

%Set up the actual scene
Rows=5; Columns=5; %The size of the figure output, which is 5*5
Box = ones(Rows, Columns);

Box(2,1) = BackGround1;
Box(2,2) = BackGround2;
Box(2,3) = BackGround3;
Box(2,4) = BackGround4;
Box(2,5) = BackGround5;
Box(3,1) = BackGround6;
Box(3,2) = BackGround7;
Box(3,3) = BackGround8;
Box(3,4) = BackGround9;
Box(3,5) = BackGround10;
Box(4,1) = BackGround11;
Box(4,3) = BackGround12;
Box(4,5) = BackGround13;
Box(4,2) = Replay; %Start Button
Box(4,4) = Exit; %Exit Button

drawScene(SDPgame, Box); %draw out the figure
xlabel("Click Replay/Exit or Press r/e; You can choose to replay the game or exit the game")
title("Endgame Page"); %The title of the created figure so I know which figure is for which

choice = ""; %Set the choice to be initially empty

while choice == "" %"" because We want the code inside the loop to continue for execute only if the choice is empty
    %This part of the code is for the user's mouse input
    [r,c,b] = getMouseInput(SDPgame); %Row, Column, and b is which key you click with by your mouth
    if ~isempty(r) && b==1 %~isempty(r) is to makesure the if user clicked on anything; and b==1 because the correct feedback will only occur when the user clic
        if r==4 && c==2
            choice="Replay"; %The Replay icon is located on (4,2), which is why the code is r==4 && c==2. r means row, c means column
        end
        if r==4 && c==4
            choice="Exit"; %The EXIT icon is located on (4,4), which is why the code is r==4 && c==4
        end
    end

    %This part of the code is for the user's keyboard input
    if choice==" " %"" because We want the code inside the loop to continue for execute only if the choice is empty. Basically, if the keyboard button is not cl
        key = getKeyboardInput(SDPgame); % As long as k is one of the keys We specify, the user's choice can be assigned the correct value.
        if strcmp(key,"r") %If the user DOUBLE clicked s, the game will start again
            choice="Replay";
        end

        if strcmp(key,"e") %If the user DOUBLE clicked s, they will exit the end game scene
            choice="Exit";
        end
    end
end

%Feedback for the user when they "made" a choice so they know what they are
%doing when the clicked the keys
if choice=="Replay"
    title("Play Again");
    xlabel("Reloading the game...");
    pause(2); %wait for 2 seconds
elseif choice=="Exit"
    title("Exit the game");
    xlabel("Closing the game...");
    pause(2);
    gameContinue = false; %Set the gameContinue to false to end the main loop.
end
end

```

```

% The User-defined function: userPlayChapterLevel
% This function waits for the player to click on an option (box) and determines whether the correct option (box) has been clicked twice.
% SDPgame: the thing that displays the game image.
% correctChoice: The coord matrix of the correct answer (like [1,4; 4,1]).
% chapterName: The chapter name that will be shown to the user when they clicked the wrong choice.

function arrayfinal = population(input, rho, cal, dirt)

for row = 3:24
    for col = 1:24
        input(row, col) = dirt(row, col);
    end
end

for row = 3:row
    input(row, 1) = 555;
    input(row, 24) = 555;
end

for col = 1:col
    input(3,col) = 555;
    input(24,col) = 555;
end
for x = 1:24
    input(2, x) = 17;
end
for x = 1:24
    input(1, x) = 18;
end

for i = 1:length(rho)
    input(rho(i),cal(i)) = 555;
end
%draw oil ceo at top consistently
input(1,12) = 159;

arrayfinal = input;
end
function success = userPlayChapterLevel(SDPgame, correctChoice, chapterName) % correctChoice is a matrix, the format is [r1, c1; r2, c2]

count = 0; %Initial count is 0
success = false; %The default result is the user will fail

while count < 2 %If the user selects the correct choice less than twice, the the game will wait for the user to click another choice (If the choice is correct).
    [r,c,b] = getMouseInput(SDPgame); %Get the player's mouse click data, r = row, c = column, and b = which mouse button was clicked.

    if b == 1 %We use b==1 because the correct feedback will only occur when the user clicks the button/choice with the left mouse button.

        %use the function ismember to check if the user clicked on the right box, represented by [r, c]
        if ismember([r, c], correctChoice, 'rows')
            count = count + 1; %If the user clicked on the right answer, the count will +1
            title('Correct! Find one more!');
        else
            title(['Sorry, your choice is incorrect in ' chapterName '. You have to replay the game.']);
            pause(4);
            return; %Will end the game, which the user will lose the game, and success = false
        end
    end
end

success = true; %If the count is 2, that means the user is success, then the user will go to the next chapter, anf success=true
end

close all;

```

currantdialoguestring =

"Hello it is me the evil big oil CEO. I am going to give you a hint. When fossil fuels like coal are burned and processed, they release CO₂, and sulfur dioxide,

excludedrow =

Columns 1 through 13

7 7 7 7 7 7 7 7 7 7 7 7 7

Columns 14 through 26

7 7 7 7 7 7 9 9 9 9 9 9 9

Columns 27 through 39

9 9 9 9 9 9 9 9 9 9 9 9 9

excludedrow =

Columns 1 through 13

7 7 7 7 7 7 7 7 7 7 7 7 7

Columns 14 through 26

7 7 7 7 7 7 9 9 9 9 9 9 9

Columns 27 through 39

9 9 9 9 9 9 9 9 9 9 9 9 9

Columns 40 through 52

11 11 11 11 11 11 11 11 11 11 11 11 11

Columns 53 through 60

11 11 11 11 11 11 11 11

excludedrow =

Columns 1 through 13

7 7 7 7 7 7 7 7 7 7 7 7 7

Columns 14 through 26

7 7 7 7 7 7 9 9 9 9 9 9 9

Columns 27 through 39

9 9 9 9 9 9 9 9 9 9 9 9 9

Columns 40 through 52

11 11 11 11 11 11 11 11 11 11 11 11 11

Columns 53 through 64

11 11 11 11 11 11 11 11 4 4 5 6

ans =

64

ans =

64

excludedrow =

Columns 1 through 13

7 7 7 7 7 7 7 7 7 7 7 7 7

Columns 14 through 26

7 7 7 7 7 7 9 9 9 9 9 9 9

Columns 27 through 39

9 9 9 9 9 9 9 9 9 9 9 9 9

Columns 40 through 52

11 11 11 11 11 11 11 11 11 11 11 11 11

Columns 53 through 64

11 11 11 11 11 11 11 11 4 4 5 6

excludedcol =

Columns 1 through 13

2 4 5 6 7 9 10 11 12 13 14 15 16

Columns 14 through 26

17	18	19	20	21	23	1	2	3	4	6	7	8
----	----	----	----	----	----	---	---	---	---	---	---	---

Columns 27 through 39

9	10	11	12	13	14	15	16	17	18	19	20	21
---	----	----	----	----	----	----	----	----	----	----	----	----

Columns 40 through 52

1	2	3	4	5	6	7	8	9	10	11	12	13
---	---	---	---	---	---	---	---	---	----	----	----	----

Columns 53 through 64

14	15	16	17	18	19	20	21	8	13	13	13
----	----	----	----	----	----	----	----	---	----	----	----

currantdialoguestring =

"When I burn petroleum to spin turbines, it releases greenhouse gasses with various negative effects on the environment."

currantdialoguestring =

"What! Wind turbines? The only negative environmental impacts of wind turbines is the disturbance to wildlife, but it also costs alot and takes a while, so I wi